SQL interview
questions & answers
SQL
SQL
ProjectPro

Explore beginner-friendly and advanced SQL interview questions with answers, syntax examples, and real-world database concepts for preparation.

Looking to land a job as a data analyst or a data scientist, SQL is a must-have skill on your resume. Everyone uses SQL to query data and perform analysis, from the biggest names in tech like Amazon, Netflix, and Google to fast-growing seed-stage startups in data. Before the world was taken over by the buzz of data science and analytics, data management still existed. Yes, you heard that right! Data was being managed, queried, and processed using a popular tool- SQL! SQL is considered the industry-standard programming language for extracting data, analyzing data, performing complex analysis, and validating hypotheses.

SQL is a highly desirable skill if you plan to become a data analyst or a data scientist. One cannot imagine having a successful career in data science or data analytics without mastering SQL.

Top 100 SQL Interview Questions for Data Analyst to Prepare in 2025

A data analyst is like a translator between the database and the other data science team members. This makes SQL a crucial skill to master before appearing for any data analyst interview. This blog covers some most commonly asked SQL practice questions for data analyst interviews, so you will be able to better equip yourself with various SQL concepts before your data analyst interview.

Basic SQL Interview Questions and Answers for Freshers

If you're starting your journey in data or software roles, SQL is one of the most essential skills to master. This guide has a list of common SQL interview questions and answers to help beginners prepare for SQL interviews with clear questions, simple answers, and easy-to-follow examples that build your confidence and understanding.

1. What is RDBMS?

RDBMS stands for Relational Database Management System. It organizes data into tables (also called relations) which are linked using keys. Examples include MySQL, PostgreSQL, and Oracle.

2. What are tables and fields in SQL?

Tables store data in rows and columns. Each column is called a field and represents a data attribute. Each row is a record.

3. What is an SQL Query?

An SQL query is a command used to perform actions like retrieving, updating, or deleting data in a database.

4. What is a schema in SQL?

A schema is a logical structure that holds database objects like tables, views, and indexes. It organizes and separates data.

5. What is a comment in SQL?

Comments are notes in SQL code for human readers. Single-line comments start with --, and multi-line comments use /* */.

6. What are SQL operators?

SQL operators are symbols or keywords used to perform operations on data. They include arithmetic, comparison, logic, and others.

7. What are the types of SQL operators?

- Arithmetic: +, -, *, /
- Comparison: =, !=, >, <, >=, <=
- Logical: AND, OR, NOT
- Others: BETWEEN, IN, LIKE

8. What are the different types of SQL queries, and how are they categorized?

SQL queries are typically categorized based on the operation they perform:

- DQL (Data Query Language): SELECT

- DDL (Data Definition Language): CREATE, ALTER, DROP, TRUNCATE

- DML (Data Manipulation Language): INSERT, UPDATE, DELETE

- DCL (Data Control Language): GRANT, REVOKE

- TCL (Transaction Control Language): COMMIT, ROLLBACK, SAVEPOINT

9. What is a JOIN in SQL?

A JOIN is used to combine rows from two or more tables based on a related column. It enables data retrieval across tables.

10. What are the main types of JOINs?

- INNER JOIN: Returns matching rows from both tables.
- LEFT JOIN: All rows from the left table, matched rows from the right.
- RIGHT JOIN: All rows from the right table, matched rows from the left.

- FULL JOIN: All rows from both tables with NULLs for unmatched rows.
- CROSS JOIN: Returns Cartesian product of two tables.

11. How do LEFT JOIN and RIGHT JOIN differ?

LEFT JOIN returns all rows from the left table and matched ones from the right. RIGHT JOIN does the opposite, returning all from the right and matched from the left.

12. What is the difference between FULL JOIN and CROSS JOIN?

A Full Outer Join is formed by combining a Left Outer and a Right Outer Join. It returns the query's WHERE clause in all rows from both tables, and it returns a NULL value if the ON condition is not met.

A cross join yields all possible combinations of all rows from both tables, generating a cartesian product.

13. What are SQL dialects? List a few examples.

SQL dialects refer to the different versions or "flavors" of SQL implemented by various database management systems. While the core SQL syntax (like SELECT, INSERT, UPDATE, etc.) is standardized by ANSI, each database system may extend it with its own custom functions, commands, and behaviors—resulting in slightly different dialects.

These dialects allow databases to optimize performance and add advanced features, but they may introduce variations in syntax, data types, and supported functions.

Examples of popular SQL dialects include:

- MySQL: Widely used in web development; supports functions like LIMIT for pagination.
- PostgreSQL: An open-source, feature-rich dialect known for standards compliance and advanced indexing.
- Microsoft SQL Server (T-SQL): Adds procedural programming features and custom functions like TOP.
- Oracle SQL (PL/SQL): Uses PL/SQL for advanced procedural logic and error handling.
- SQLite: Lightweight and file-based; supports a subset of standard SQL.

Understanding the dialect you're working with is important for writing compatible and efficient queries.

14. What is an alias in SQL?

An alias gives a table or column a temporary name to make queries more readable.

Example:

SELECT name AS employee_name FROM Employees;

15. What statements are commonly used with SELECT?

- WHERE
- ORDER BY
- GROUP BY
- HAVING
- LIMIT / TOP (depends on DBMS)

16. What are constraints in SQL, and why are they used?

Constraints enforce rules on data:

- NOT NULL ensures a column can't be NULL.
- UNIQUE to check all values must be different.
- PRIMARY KEY helps to uniquely identify a row.
- FOREIGN KEY enforces referential integrity.
- CHECK ensures values meet a condition.
- DEFAULT provides default values.

17. What is a subquery in SQL, and what are its types?

A subquery is a query inside another query.

Types:

- Scalar Subquery returns a single value.
- Column Subquery returns a single column.
- Row Subquery returns a single row.
- Table Subquery returns a result set.
- Correlated Subquery depends on the outer query.

18. What are relationships in SQL?

Relationships link tables using primary and foreign keys. Types include one-to-one, one-to-many, and many-to-many.

19. What is a NULL value in SQL?

NULL represents missing or unknown data. It is not the same as zero or an empty string.

20. What is the difference between SQL and NoSQL?

- SQL is structured and table-based (relational).
- NoSQL supports unstructured or semi-structured data (e.g., documents, key-value pairs). SQL is better for complex queries and consistency; NoSQL offers flexibility and scalability.

21. What are the challenges of working with SQL databases?

- Scalability for big data can be limited
- Complex joins can affect performance
- Schema changes require careful planning
- Maintaining consistency in distributed systems

22. What are window functions in SQL? Provide examples.

Window functions operate across rows related to the current row.

Example:

```
SELECT EmployeeID, Salary,
    RANK() OVER (ORDER BY Salary DESC) AS SalaryRank
FROM Employees;
```

Other functions: ROW_NUMBER(), LAG(), LEAD(), SUM() OVER(...)

23. What are Common Table Expressions (CTEs) in SQL?

CTEs are temporary result sets using the WITH clause.

Example:

```
WITH HighSales AS (
  SELECT DepartmentID, SUM(Sales) AS TotalSales
  FROM SalesData
```

```
    GROUP BY DepartmentID
)
SELECT * FROM HighSales WHERE TotalSales > 50000;
```

Normalization organizes data to reduce redundancy.

Types

- 1NF: Atomic columns.
- 2NF: 1NF + no partial dependency.
- 3NF: 2NF + no transitive dependency.
- BCNF, 4NF, 5NF: Higher forms for complex scenarios.

24. What is denormalization and how does it differ from normalization?

Denormalization adds redundancy to improve read performance.

Normalization = data integrity, less redundancy.

Denormalization = faster reads, more redundancy.

25. What is indexing in SQL and how does it improve performance?

Indexing speeds up queries by avoiding full table scans.

- Clustered Index: Sorts and stores data rows.
- Non-Clustered Index: Uses pointers to data.

Useful for WHERE, JOIN, ORDER BY but can slow down writes.

26. What is the difference between a Primary Key, Foreign Key, and Unique Key in SQL? Provide examples.

Primary Key

- A column or a set of columns that uniquely identifies each row in a table.
- It cannot contain NULLs and must be unique.
- Each table can have only one primary key.

Example:

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
```

```
    Name VARCHAR(100)
);
```

Unique Key

- Ensures that all values in a column are distinct, like a primary key.
- Unlike a primary key, a table can have multiple unique keys.
- A unique key can contain NULLs (but only one NULL in most databases).

Example:

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE
);
```

Foreign Key

- A column that creates a relationship between two tables.
- It references the primary key of another table, enforcing referential integrity.
- It can have duplicate values and may contain NULLs (unless specified otherwise).

Example:

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Feature	Primary Key	Unique Key	Foreign Key
Uniqueness	Yes	Yes	No (can have duplicates)
NULL Allowed	No	Yes (usually one)	Yes
Count per Table	One	Multiple	Multiple
Purpose	Row Identifier	Enforce uniqueness	Define relationship between tables

Besides these, here are a few more sql questions for interview preparation by Himanshu Bansal, Data Analytics Leader with over 12 years of experience.

SQL Technical Interview Questions

This section covers core technical questions that test your understanding of SQL fundamentals, database design, constraints, relationships, normalization, and key concepts. Ideal for candidates expected to demonstrate both theoretical and practical knowledge of relational databases.

27. What is a function in SQL?

A function in SQL performs a specific task and returns a single value or a result set. Functions can be built-in or user-defined.

28. What are the types of functions in SQL?

- Aggregate functions
- Scalar functions
- Case/Character manipulation functions

29. What are aggregate functions?

They operate on a set of values and return a single value. Examples: COUNT(), SUM(), AVG(), MAX(), MIN().

30. What are scalar functions?

Scalar functions operate on a single value and return a single value. Examples: UPPER(), LOWER(), ROUND(), LEN().

31. What are case manipulation functions?

These change the case of characters:

- UPPER(): Converts to uppercase

- LOWER(): Converts to lowercase
- INITCAP(): Capitalizes first letter (DBMS-dependent)

32. What are character manipulation functions?

Functions that work with text:

- LENGTH()
- SUBSTRING()
- CONCAT()
- TRIM()

33. What is the difference between local and global variables?

- Local variables are declared inside a block/procedure and are only accessible there.
- Global variables are accessible across procedures and sessions.

34. What is the difference between SQL and PL/SQL?

- SQL is a query language for accessing data.
- PL/SQL is Oracle's procedural extension for SQL, allowing loops, conditions, and error handling.

35. What is the difference between LEFT JOIN and OUTER JOIN?

LEFT JOIN is a type of OUTER JOIN. OUTER JOIN includes LEFT, RIGHT, and FULL joins. LEFT JOIN returns all rows from the left and matched ones from the right.

36. What is indexing in SQL and why is it used?

Indexes improve query speed by allowing faster searches. They're similar to book indexes.

37. What is a stored procedure? How is it different from a function?

- A stored procedure performs tasks and may or may not return a value.
- A function must return a value and is used in expressions.

38. What is the default data ordering with ORDER BY? How do you change it?

ORDER BY sorts data in ascending order by default. Use DESC for descending.

SELECT * FROM Students ORDER BY Age DESC;

39. What are set operators in SQL?

Set operators combine results from multiple SELECT queries:

- UNION
- UNION ALL
- INTERSECT
- MINUS (or EXCEPT)

40. Which operator is used for pattern matching in SQL?

The LIKE operator is used with % (wildcard) for pattern matching.

SELECT * FROM Students WHERE Name LIKE 'A%';

41. What is a composite primary key?

A primary key made of two or more columns.

PRIMARY KEY (OrderID, ProductID)

42. What is a view and why is it used?

A view is a virtual table created from a query. It's used to simplify queries, enhance security, and present data differently.

43. Can a view be based on another view?

Yes, views can be created on top of other views.

44. What are the types of SQL relationships?

- One-to-One
- One-to-Many
- Many-to-Many

45. What are possible values of a BOOLEAN field?

Typically TRUE, FALSE, and NULL (depends on DBMS).

46. What is the difference between renaming a column and giving an alias?

- Renaming: permanent change to schema.
- Alias: temporary name for a query using AS.

47. What are nested and correlated subqueries?

- Nested: runs independently of the outer query.
- Correlated: depends on outer query values and runs for each row.

48. What are clustered and non-clustered indexes?

- Clustered: sorts the actual data rows.
- Non-clustered: maintains a separate structure with pointers to the data.

49. What is the CASE() function in SQL?

CASE is used for conditional logic in queries.

```
SELECT Name,
    CASE WHEN Age < 18 THEN 'Minor' ELSE 'Adult' END AS Category
FROM Students;
```

50. What is the difference between DELETE and TRUNCATE?

- DELETE: removes specific rows, can be rolled back.
- TRUNCATE: removes all rows, faster, can't be rolled back.

51. What is the difference between DROP and TRUNCATE?

- DROP: deletes the table structure and data.
- TRUNCATE: removes all data but keeps the structure.

52. What is the difference between HAVING and WHERE clauses?

- WHERE: filters rows before aggregation.
- HAVING: filters groups after aggregation.

```
SELECT department, COUNT(*)
FROM Employees
GROUP BY department
HAVING COUNT(*) > 5;
```

SQL Coding Interview Questions by Tech Giants

Top tech companies often ask SQL questions that go beyond theory and focus on real-world data problems. In this section, you'll explore coding-style SQL questions similar to those asked by companies like Google, Amazon, and Meta, helping you practice logical thinking and query optimization.

53. Between 2019-02-01 and 2019-05-01, find the customer with the highest overall order cost. If a consumer placed multiple orders on the same day, calculate the order expenses daily. The output must include their initial name, the total cost of the items, and the date.

Also, assume that each first name in the dataset is distinct.

```
SELECT first_name,
       sum(total_cost) AS total_cost,
       order_date
FROM orders od
LEFT JOIN customers cst ON od.cust_id = cst.id
WHERE order_date BETWEEN '2019-02-1' AND '2019-05-1'
GROUP BY first_name,
         order_date
HAVING sum(total_cost) =
  (SELECT max(total_cost)
   FROM
     (SELECT sum(total_cost) AS total_cost
      FROM orders
      WHERE order_date BETWEEN '2019-02-1' AND '2019-05-1'
      GROUP BY cust_id,
               order_date)
```

54. For each city, determine the number of orders, the number of clients, and the total cost of orders. The output must include only cities with at least five orders and all clients in each city, even those who did not place an order.

Display each computation along with the name of the city it corresponds to.

```
SELECT customers.city,
    count(DISTINCT orders.id) AS orders_per_city,
    count(DISTINCT customers.id) AS customers_per_city,
    sum(orders.total_order_cost) AS orders_cost_per_city
FROM customers
LEFT JOIN orders ON customers.id = orders.cust_id
GROUP BY customers.city
HAVING count(orders.id) >=5
SELECT customers.city,
    count(DISTINCT orders.id) AS orders_per_city,
    count(DISTINCT customers.id) AS customers_per_city,
    sum(orders.total_order_cost) AS orders_cost_per_city
FROM orders
RIGHT JOIN customers ON orders.cust_id = customers.id
GROUP BY customers.city
HAVING count(orders.id) >=5
```

55. In a marathon, gun time is measured from the moment the race is officially started, whereas net time is measured from the moment a runner crosses the starting line. The timestamps for both variables are in seconds.

You must determine whether the interval between the two times for male and female runners is different. Calculate the average absolute difference between a gun and net time first. Sort the results by the genders- male and female. The absolute difference between the two values should be output.

```
WITH male_avg_diff AS
  (SELECT AVG(ABS(gun_time - net_time)) AS abs_diff
```

```
  FROM marathon_male),
    female_avg_diff AS
 (SELECT AVG(ABS(gun_time - net_time)) AS abs_diff
  FROM marathon_female)
SELECT ABS(m.abs_diff - f.abs_diff) difference
FROM male_avg_diff m
CROSS JOIN female_avg_diff f
```

56. Determine the number of employees from Arizona, California, and Hawaii. Arrange all city data alphabetically by the first name. Also, Arizona, California, and Hawaii should be the output column headers.

Assume that each city has a different number of employees.

```
WITH arizona AS
 (SELECT first_name AS aname,
        ROW_NUMBER() OVER (
                ORDER BY first_name) AS arn
  FROM employee
  WHERE city='Arizona' ),
    california AS
 (SELECT first_name AS cname,
        ROW_NUMBER() OVER (
                ORDER BY first_name) AS crn
  FROM employee
  WHERE city='California' ),
    hawaii AS
 (SELECT first_name AS hname,
        ROW_NUMBER() OVER (
                ORDER BY first_name) AS hrn
```

```
  FROM employee
  WHERE city='Hawaii' )
SELECT aname AS arizona,
     cname AS california,
     hname AS hawaii
FROM hawaii
FULL OUTER JOIN california ON hrn=crn
FULL OUTER JOIN arizona ON COALESCE(hrn, crn) =arn;
```

Download [ProjectPro's Ultimate SQL cheatsheet]() for quick revision and interview prep.

SQL Query Interview Questions and Answers

Often, data analysts are asked SQL queries interview questions. This section will list some query-based SQL interview questions for data analysts.

57. Consider the following query-

```
SELECT column, AGG_FUNC(column_or_expression), …
  FROM a_table
 INNER JOIN some_table
     ON a_table.column = some_table.column
 WHERE a_condition
 GROUP BY column
HAVING some_condition
 ORDER BY column
 LIMIT 5;
```

What is the order in which SQL executes the clauses? Choose the best option from the choices below:

SELECT, FROM, WHERE, GROUP BY

FROM, WHERE, HAVING, SELECT, LIMIT

SELECT, FROM, INNER JOIN, GROUP BY

FROM, SELECT, LIMIT, WHERE

Option 2 is the right answer.

SQL server retrieves the data from the tables (FROM and INNER JOIN)

Processes it (WHERE)

Combines the information (GROUP BY)

Filters the gathered data (HAVING)

Chooses which columns and expressions to show (SELECT)

Sorts the remaining information (ORDER BY)

Limits the outcomes (LIMIT)

Now, consider the following tables-

Emp_Details-

EmpId	FullName	ManagerId	DateofJoining	City
121	Emma Greene	321	01/05/2018	London
321	Janice Linn	657	13/07/2019	Sydney
421	William Scott	329	25/11/2022	California

Emp_Salary-

EmpId	Project	Salary	Variables
121	P1	7000	0
321	P2	12000	2000
421	P1	8000	1000

58. Create a SQL query to obtain a list of all the employees who are also managers in the EmpDetails table.

```
SELECT DISTINCT E.FullName

FROM Emp_Details E

INNER JOIN Emp_Details M

ON E.EmpID = M.ManagerID;
```

59. Create a SQL query to retrieve duplicate records from EmployeeDetails (neglecting the primary key, EmpId).

```
SELECT FullName, ManagerId, DateOfJoining, City, COUNT(*)

FROM Emp_Details

GROUP BY FullName, ManagerId, DateOfJoining, City

HAVING COUNT(*) > 1;
```

60. Without creating a temporary table, write a SQL query to eliminate duplicates from a table.

```
DELETE E1 FROM Emp_Details E1

INNER JOIN Emp_Details E2

WHERE E1.EmpId > E2.EmpId

AND E1.FullName = E2.FullName

AND E1.ManagerId = E2.ManagerId

AND E1.DateOfJoining = E2.DateOfJoining

AND E1.City = E2.City;
```

61. Create a SQL query to determine the table's nth highest income.

```
SELECT TOP 1 Salary

FROM (

    SELECT DISTINCT TOP N Salary

    FROM Employee

    ORDER BY Salary DESC

    )
```

```
ORDER BY Salary ASC;
```

Using the limit clause in MySQL-

```
SELECT Salary

FROM Employee

ORDER BY Salary DESC LIMIT N-1,1;
```

62. Without employing the TOP/limit keyword, write a SQL query to find the 3rd highest income from a table.

```
SELECT Salary

FROM Emp_Salary Emp1

WHERE 2 = (

        SELECT COUNT( DISTINCT ( Emp2.Salary ) )

        FROM Emp_Salary Emp2

        WHERE Emp2.Salary > Emp1.Salary

    )
```

63. Create a SQL query that only returns even rows from the table.

If you have an auto-increment field, such as EmpId, you can use the query below-

```
SELECT * FROM Emp_Details

WHERE MOD (EmpId, 2) = 0;
```

If you don't have such a field, you can use the queries below-

Using SQL server's Row_number ensures that the remainder is 1 when divided by 2.

```
SELECT E.EmpId, E.Project, E.Salary

FROM (

   SELECT *, Row_Number() OVER(ORDER BY EmpId) AS RowNumber

   FROM Emp_Salary

) E

WHERE E.RowNumber % 2 = 0;
```

Using a MySQL user-defined variable-

```
SELECT *
FROM (
    SELECT *, @rowNumber := @rowNumber+ 1 rn
    FROM Emp_Salary
    JOIN (SELECT @rowNumber:= 0) r
   ) t
WHERE rn % 2 = 0;
```

64. Create a SQL query to retrieve the top n records.

Using LIMIT in MySQL-

```
SELECT *
FROM EmpSalary
ORDER BY Salary DESC LIMIT N;
```

Using the TOP command in SQL Server-

```
SELECT TOP N *
FROM Emp_Salary
ORDER BY Salary DESC;
```

65. Create a SQL query to retrieve only the table's odd rows.

If you have an auto-increment field, such as EmpId, you can use the query below-

```
SELECT * FROM Emp_Details
WHERE MOD (EmpId, 2) <> 0;
```

If you don't have such a field, you can use the queries below-

Using SQL server's Row_number and ensuring that the remainder is 1 when divided by 2-

```
SELECT E.EmpId, E.Project, E.Salary
FROM (
   SELECT *, Row_Number() OVER(ORDER BY EmpId) AS RowNumber
   FROM Emp_Salary
```

```
) E

WHERE E.RowNumber % 2 = 1;
```

Using a MySQL user-defined variable-

```
SELECT *

FROM (

    SELECT *, @rowNumber := @rowNumber+ 1 rn

    FROM Emp_Salary

    JOIN (SELECT @rowNumber:= 0) r

   ) t

WHERE rn % 2 = 1;
```

Advanced SQL Interview Questions

Designed for experienced professionals, this section explores advanced SQL topics such as window functions, CTEs, complex subqueries, performance tuning, indexing strategies, and database optimization techniques. These SQL interview questions for experienced professionals assess both depth and breadth of SQL expertise.

66. What is a Common Table Expression (CTE), and how does it differ from a subquery?

A Common Table Expression (CTE) is a temporary result set defined within the execution scope of a single SELECT, INSERT, UPDATE, or DELETE statement. It improves query readability and can be used recursively, unlike subqueries.

Example:

```
WITH DepartmentSales AS (

   SELECT DepartmentID, SUM(Sales) AS TotalSales

   FROM SalesData

   GROUP BY DepartmentID

)

SELECT * FROM DepartmentSales WHERE TotalSales > 100000;
```

67. What are the ACID properties in database systems?

- Atomicity – all operations in a transaction complete or none do.
- Consistency – ensures database rules are followed.
- Isolation – transactions don't interfere with each other.
- Durability – once a transaction is committed, it stays committed.

68. How do indexes improve query performance, and what are the types of indexes?

Indexes speed up data retrieval. Clustered indexes sort data physically. Non-clustered indexes store pointers to data.

Clustered – one per table, determines physical order.

Non-clustered – multiple allowed, uses pointers.

69. What is the difference between DELETE, TRUNCATE, and DROP statements in SQL?

DELETE – removes specific rows; can be rolled back.

TRUNCATE – removes all rows; faster, cannot be rolled back in some systems.

DROP – removes entire table structure.

70. What is a materialized view, and how does it differ from a regular view?

A regular view is virtual and doesn't store data. A materialized view stores data physically and can be refreshed for better performance on complex queries.

71. How do you identify and remove duplicate rows in a SQL table?

To find duplicates:

```
SELECT Column1, Column2, COUNT(*) FROM TableName GROUP BY Column1, Column2
HAVING COUNT(*) > 1;
```

To delete duplicates:

```
DELETE FROM TableName

WHERE ID NOT IN (

   SELECT MIN(ID)

   FROM TableName

   GROUP BY Column1, Column2
```

```
);
```

72. What is a recursive CTE, and when would you use it?

Recursive CTEs call themselves to iterate through hierarchical data, such as organizational charts.

Example:

```
WITH EmployeeHierarchy AS (
    SELECT EmployeeID, ManagerID, 0 AS Level
    FROM Employees
    WHERE ManagerID IS NULL
    UNION ALL
    SELECT e.EmployeeID, e.ManagerID, eh.Level + 1
    FROM Employees e
    INNER JOIN EmployeeHierarchy eh ON e.ManagerID = eh.EmployeeID
)
SELECT * FROM EmployeeHierarchy;
```

73. How can you optimize a slow-running SQL query?

Use EXPLAIN to review query plans.

Add indexes on frequently filtered/sorted columns.

Avoid SELECT *.

Filter early using WHERE.

Use LIMIT to reduce row count.

Update database statistics.

Optimize joins and avoid unnecessary nesting.

74. With the help of an example, explain Equi join.

Equi join is a type of join that occurs when you join two or more tables using the 'equal to' operator. Only the condition equal to(=) between the columns in the table needs to be focused on.

Example: From Employee a, Employee b, select a.Emp name,b.Dept name, where a.Dept ID=b.Dept ID.

75. With an example, explain the Right Outer Join.

This join is suitable when the user wants all of the records from the Right table (Second table) and only equal or matching records from the First or Left table. You can refer to the records that don't match as null records.

Example: Select p1.col1, p2.col2,...p 'n'col 'n.' from table1 p1, table2 p2 where p1.col(+)=p2.col;

76. In SQL, how do you alter the name of a column?

In MYSQL, this is the command to alter the name of a column:

ALTER TABLE table_name CHANGE old_colname new_colname char(50);

You'll begin by using the ALTER TABLE commands, followed by the table's name. After that, you'll use the CHANGE keyword to provide the column's original name, followed by the name to which you'd like to rename it.

77. Explain Natural Join with the help of an example.

A natural join establishes an implicit join clause between two tables based on shared characteristics. In both tables, a common characteristic has the same name. Unlike an equi join, a Natural join does not require a comparison operator.

Consider the following example of a Natural join. Employee and Department are two tables with the same field, 'EmpNo.'

Employee

EmpId	EmpName	City
123	Adam	San Fransisco
456	James	New York
789	Duke	Paris

Department

DeptId	DeptName	EmpId
342	HR	123
145	Sales	456
760	Admin	789

Syntax:

```
SELECT Employees.EmpName, Department.DeptName

FROM Department Natural JOIN Employees
```

SQL Interview Questions and Answers for Data Analysts

Explore these carefully curated SQL interview questions tailored for experienced data analytics professionals.

78. Find the top ten users who have traveled the furthest. List their names as well as the overall distance traveled. (Consider the table names- cab_users and cab_rides_log)

```
SELECT name, traveled_distance

FROM

  (SELECT cu.name,

      SUM(cr.distance) AS traveled_distance,

      rank () over (order by SUM(cr.distance) desc) as rank

  FROM cab_users AS cu

  INNER JOIN cab_rides_log AS cr ON cu.id = cr.user_id

  GROUP BY cu.name

  ORDER BY traveled_distance DESC

  ) sq
```

```
WHERE rank <= 10
```

Suppose a company has created a search algorithm that will scan through user comments and display the search results to the user. We have a table containing the search result that the user clicked on ('notes' column), the user's search query, and the final search ranking returned for the specific comment to evaluate the algorithm's performance.

Since these remarks were what the user was looking for, the higher the position, the better. Create a query that analyses the search algorithm's performance against each user query. (Use the table search_results)

```
SELECT t.result_id,
    t.query,
    CASE
      WHEN t.check = FALSE THEN 1
      WHEN t.check = TRUE
          AND t.position >= 10 THEN 2
      WHEN t.check = TRUE
          AND (t.position BETWEEN 6 AND 9) THEN 3
      WHEN t.check = TRUE
          AND (t.position BETWEEN 4 AND 5) THEN 4
      WHEN t.check = TRUE
          AND t.position <=3 THEN 5
    END AS rating
FROM
  (
SELECT query,
        result_id,
        position,
        notes,
        (regexp_replace(notes, '[^\w]+',' ','g') ilike concat
('% ', query,' %')) AS check
```

```
  FROM search_results
 ) t
```

79. Create a query that returns cities with housing prices greater than the national average. Generate a list of city names. (Use the table xyz_transactions).

```
SELECT city
FROM xyz_transactions a
GROUP BY city
HAVING avg(a.mkt_price) >
  (SELECT avg(mkt_price)
   FROM xyz_transactions)
ORDER BY city ASC
```

80. Determine each user's email activity rank. The total number of emails sent determines the email activity rank. The user with the most emails sent receives a rank of one, and so on. Display the user's name, the total number of emails they've received, and their activity rank. Arrange records in descending order based on the total number of emails. You should sort individuals with the same rank score alphabetically.

Even if numerous users have the same quantity of emails, return a unique value (i.e., a unique percentile) in your rankings. (Use the gmail_emails table)

```
SELECT  from_user,
        count(*) as total_emails,
        row_number() OVER ( order by count(*) desc)
   FROM gmail_emails
   GROUP BY from_user
   order by 3, 1
```

81. Determine the difference between NVL and NVL2 functions.

NVL(exp1, exp2) and NVL2(exp1, exp2, exp3) check if the value exp1 is null. If exp1 is not null, the NVL(exp1, exp2) function returns the value of exp1; otherwise, it returns the value of exp2. However, exp2 must be of the same data type as exp1. If exp1 is not null, the NVL2(exp1, exp2, exp3) function returns exp2; otherwise, it returns the value of exp3.

82. List the various kinds of SQL sandboxes?

There are three types of SQL sandboxes-

1. Safe Access Sandbox- In this environment, users can conduct SQL operations such as creating stored procedures, triggers, etc. but cannot access memory or create files.

2. Sandbox for External Access- Users can access files without having the right to modify memory allocation.

3. Unsafe Access Sandbox- This is a collection of untrusted routines that allow a user to access memory.

83. What are the differences between COALESCE() and ISNULL()?

COALESCE() takes two or more parameters, and while you can apply as many as you like, it only returns the first non-NULL parameter.

ISNULL() only takes two arguments.

The first parameter is tested for a NULL value, and if it is, the second parameter is returned; if not, the first parameter is returned.

84. Compare the 'Having' and 'Where' clauses in detail.

WHERE Clause	HAVING Clause
• The WHERE clause sets the conditions that particular records must meet for a query to select them. It is not required to include the GROUP by clause.	• Without the GROUP BY clause, the HAVING clause will not execute.
• The WHERE clause chooses rows before grouping.	2. The HAVING clause chooses rows once grouping is done.
• Aggregate functions aren't permitted in the WHERE clause.	3. You can use aggregate functions in the HAVING clause.

• The WHERE clause is used before the GROUP BY clause to enforce a condition on the SELECT statement and the single row function.	4. The HAVING clause is used after the GROUP BY clause in the query to put a constraint on the GROUP Function.
• E.g.- SELECT Column, AVG(Column_name) FROM Table_name WHERE Column > value GROUP BY Column_name	5. SELECT Column, AVG(Column_name) FROM Table_name WHERE Column > value GROUP BY Column_name Having column_name>or<value

85. What is meant by cte in SQL Server?

Common Table Expressions (CTEs) are expressions used to build temporary output tables from which data can be obtained and used. CTEs can also be used with Insert, Update, and Delete statements. A CTE with a SELECT statement has the following standard syntax-

WITH RESULT AS

(SELECT COL1, COL2, COL3

FROM EMPLOYEE)

SELECT COL1, COL2 FROM RESULT

86. Distinguish between MongoDB and MySQL.

MongoDB	MySQL
• MongoDB is the right choice when you require high data availability with automatic, quick, and immediate data recovery.	• MYSQL will make the setup simple and low-maintenance if you're just getting started and don't expect your database to expand much.

• If you plan to scale up in the future, MongoDB includes a built-in sharding option.	2. If you need exceptional performance on a tight budget, MySQL is the way to go.
• If your schema is unstable and you want to save on schema migration, you should work with MongoDB.	3. If you have a stable schema and data structure, you should choose MySQL.
• If you don't have access to a database administrator, MongoDB is the preferable option.	4. If you need a handful of transactions per second, MySQL is the way to go.
• MongoDB is the best choice if most of your services are cloud-based.	5. MySQL is the best DBMS to use if data security is a high priority.

87. In SQL Server, how can you build a stored procedure?

A Stored Procedure is a commonly-used SQL query. Queries like SELECT queries, which are frequently used to fetch a data set from a database, can be recorded as a Stored Procedure. The SQL query saved within the Stored Procedure is executed when the Stored Procedure is called.

To construct a Stored Procedure, use the following syntax:

CREATE PROCEDURE PROCEDURE_NAME

AS

SQL_QUERY (Write your frequently used query here)

GO;

SQL Server supports both user-defined and built-in stored procedures. Also, you can provide a variety of parameters to a Stored Procedure.

88. How do you insert a single record into a table?
You can insert one row at a time using the INSERT INTO statement followed by the column values.

```
INSERT INTO employees (id, name, department)

VALUES (101, 'Alice', 'HR');
```

89. How can you delete specific rows from a table?
Use the DELETE statement along with a WHERE clause to target which rows should be removed.

```
DELETE FROM employees

WHERE department = 'HR';
```

90. How do you add a new column to an existing table?
Use the ALTER TABLE command along with ADD to introduce a new column.

```
ALTER TABLE employees

ADD join_date DATE;
```

91. How do you rename a column in SQL?
Use the ALTER TABLE command combined with RENAME COLUMN. This syntax varies by database system.

```
ALTER TABLE employees

RENAME COLUMN join_date TO hire_date;
```

92. How do you remove a column from a table?
You can drop a column using the ALTER TABLE command with DROP COLUMN.

```
ALTER TABLE employees

DROP COLUMN hire_date;
```

93. How do you select only even or only odd rows based on an ID column?
Use the modulo operator % or the MOD() function depending on the SQL dialect.

```
-- Even rows

SELECT * FROM employees

WHERE id % 2 = 0;

-- Odd rows

SELECT * FROM employees
```

WHERE id % 2 = 1;

94. How can you avoid retrieving duplicate entries in a query?
Use the DISTINCT keyword with SELECT to ensure only unique rows are returned.

SELECT DISTINCT department FROM employees;

95. How do you insert multiple rows at once into a table?
Use one INSERT INTO statement with several sets of values.

INSERT INTO employees (id, name, department)

VALUES (102, 'Bob', 'Finance'),

(103, 'Carol', 'IT'),

(104, 'Dave', 'Marketing');

96. How can you find the nth highest salary or value in a table?
Use ORDER BY with LIMIT and OFFSET. To get the 4th highest salary:

SELECT salary FROM employees

ORDER BY salary DESC

LIMIT 1 OFFSET 3;

97. How do you find entries that start with a particular letter in a text field?
Use the LIKE operator with a wildcard %.

SELECT * FROM employees

WHERE name LIKE 'A%';

This will match names like 'Alice', 'Andrew', etc.

98. How do you find the last inserted or highest ID in a table?
Use ORDER BY along with LIMIT to retrieve the last or maximum ID.

SELECT id FROM employees

ORDER BY id DESC

LIMIT 1;

99. How can you randomly select rows from a table?
Use ORDER BY in combination with a random function like RAND() or RANDOM().

SELECT * FROM employees

ORDER BY RAND()

```
LIMIT 3;
```

This will return 3 random rows.

100. How would you identify and handle gaps in a sequential ID column in a large transactional table?

To identify missing values in a sequential ID column (like order numbers), you can use a combination of a numbers generator and an OUTER JOIN:

```
WITH seq AS (
    SELECT generate_series(1, (SELECT MAX(order_id) FROM orders)) AS expected_id
)
SELECT expected_id
FROM seq
LEFT JOIN orders ON seq.expected_id = orders.order_id
WHERE orders.order_id IS NULL;
```

This query generates a sequence of expected IDs and finds those not present in the actual table. To handle the gaps, options include:

- Logging and investigating missing IDs due to deleted or failed transactions.
- Reusing them (rarely advisable for audit-sensitive systems).
- Leaving gaps untouched and using surrogate keys to avoid business impact.

Next Steps: Preparing for Your Data Analyst Interview with ProjectPro!

One of the best ways to prepare for a data analyst job interview is to work with projects that test your hands-on knowledge on diverse data analyst skills like SQL, Excel, Python, and others. If you're hustling to gain that hands-on experience before your interview, we've got you covered.

Check out ProjectPro's solved end-to-end big data and analytics projects to gain some hands-on experience and build a data analytics portfolio of various data analyst skills that will get you hired. Here is a series of projects that might be helpful:

- SQL Project for Data Analysis using Oracle Database-Part 1
- SQL Project for Data Analysis using Oracle Database-Part 2
- SQL Project for Data Analysis using Oracle Database-Part 3

- SQL Project for Data Analysis using Oracle Database-Part 4
- SQL Project for Data Analysis using Oracle Database-Part 5
- SQL Project for Data Analysis using Oracle Database-Part 6
- SQL Project for Data Analysis using Oracle Database-Part 7